MediaGuardX

Technical Documentation

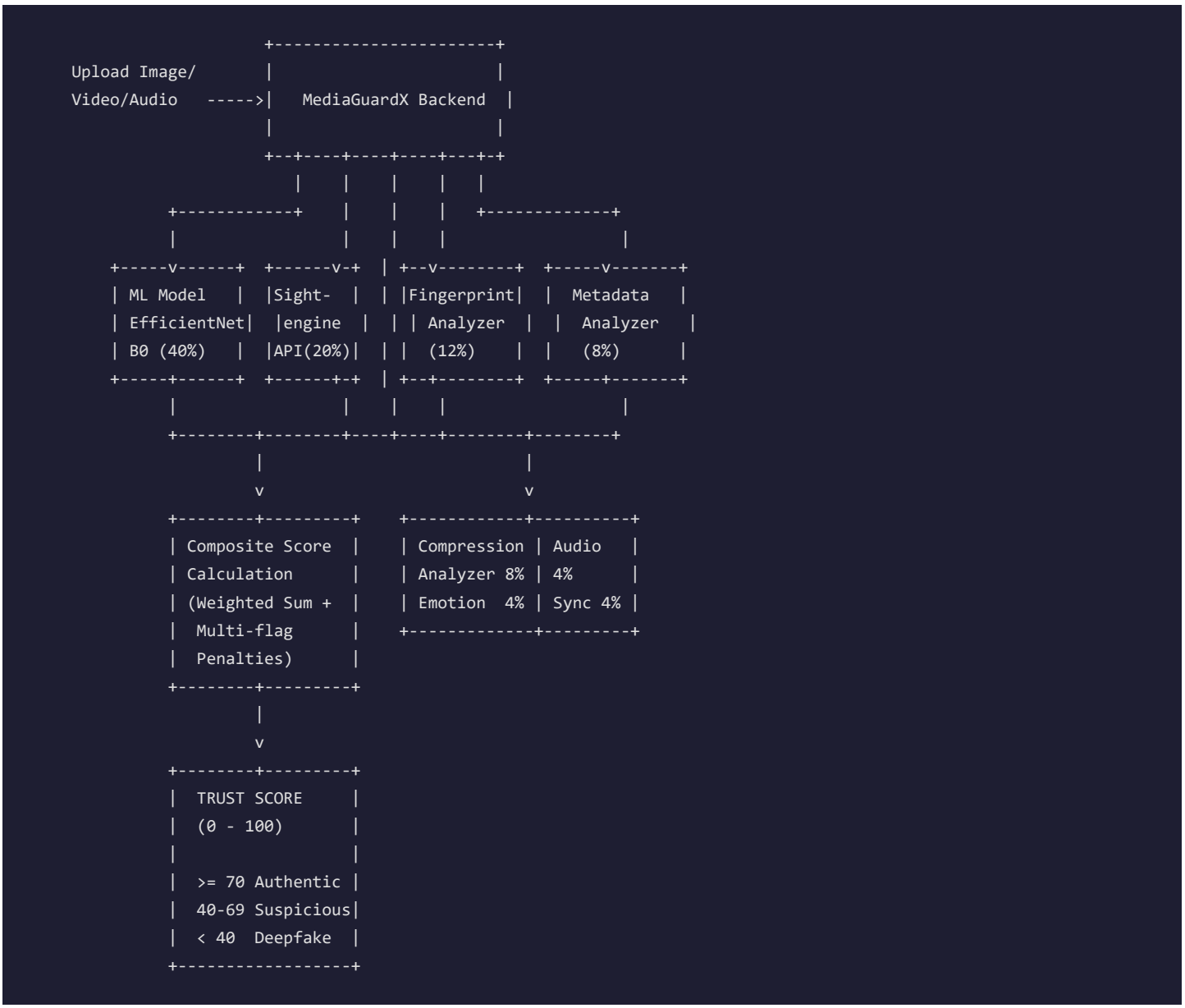
Version 2.0.0 | February 2026

Internal Technical Reference

1. System Overview

MediaGuardX is a deepfake detection platform that combines a custom-trained deep learning model with a third-party API and six heuristic analyzers to produce a composite Trust Score for any uploaded media (image, video, or audio).

The system answers one question: "How likely is this media authentic (unmanipulated)?"



2. Detection Pipeline Architecture

When a user uploads media, the following happens concurrently:

Step 1 — File Upload and Validation

- File is saved to the server using streaming chunks (8KB at a time) to prevent memory exhaustion
- Maximum file size: 50 MB
- Supported types: Images (JPEG, PNG, WebP, GIF, BMP), Videos (MP4, AVI, MOV, MKV, WebM), Audio (MP3, WAV, OGG, FLAC, M4A)

Step 2 — Parallel Analysis

All 7 analyzers run concurrently using Python's asyncio.gather():

Order	Analyzer	Runs On
1	Sightengine API	All media types
2	Metadata Analyzer	All media types
3	Fingerprint Analyzer	Images, Videos
4	Compression Analyzer	All media types
5	Audio Analyzer	Audio, Video only
6	Emotion Mismatch	Video only
7	Lip-Sync Analyzer	Video only

After all return, the ML Model runs separately (requires the saved file path).

Step 3 — Composite Score Calculation

All analyzer results are combined using weighted averaging with penalty adjustments.

Step 4 — Grad-CAM Heatmap Generation

For images: generates a visual explanation showing which regions the ML model focused on.

Step 5 — Storage and Response

Results are stored in Supabase (PostgreSQL) and returned to the frontend.

---

3. Trust Score Generation

3.1 Weighted Composite Formula

```
Trust Score = (sum of each analyzer_score x weight) / total_weight
```

Analyzer Weights:

Analyzer	Weight	Role
ML Model (EfficientNet-B0)	0.40 (40%)	Primary detector — our trained neural network
Sightengine API	0.20 (20%)	Secondary detector — third-party deep learning API
Fingerprint Analyzer	0.12 (12%)	Supporting — frequency-domain GAN artifact detection
Metadata Analyzer	0.08 (8%)	Supporting — EXIF and timestamp analysis
Compression Analyzer	0.08 (8%)	Supporting — JPEG artifact and platform detection
Audio Analyzer	0.04 (4%)	Supporting — voice cloning detection
Emotion Analyzer	0.04 (4%)	Supporting — face-audio emotion mismatch
Sync Analyzer	0.04 (4%)	Supporting — lip-sync correlation
Total	1.00 (100%)	

3.2 Score Normalization

Each analyzer produces a score in the range 0-100, where:

- 100 = fully authentic (no anomalies detected)
- 0 = definitely manipulated

Some analyzers produce inverted scores (e.g., audio "clone likelihood" is high = suspicious), which are inverted before combining: trust = 100 - suspicion_score.

3.3 Multi-Flag Penalty System

After computing the weighted average, additional penalties apply:

Condition	Penalty
3 or more heuristic analyzers score below 70	-15 points
2 heuristic analyzers score below 70	-8 points
Neither ML model nor Sightengine available	Cap score at 65 (maximum)

Rationale: If multiple independent analyzers detect problems, the evidence of manipulation is stronger than any single detector alone. The multi-flag penalty prevents a high ML model score from overriding consensus among heuristic analyzers.

3.4 Example Calculation

Scenario: Uploading a StyleGAN-generated fake face image.

Analyzer	Sub-Score	Weight	Contribution
ML Model	15.5 (fake)	0.40	6.20
Sightengine	25.0	0.20	5.00
Fingerprint	17.5 (GAN detected)	0.12	2.10
Metadata	80.0 (missing EXIF = -20)	0.08	6.40
Compression	80.0	0.08	6.40
Audio	100.0 (N/A for images)	0.04	4.00
Emotion	100.0 (N/A for images)	0.04	4.00
Sync	100.0 (N/A for images)	0.04	4.00
Total	1.00	38.10	

Multi-flag check: Fingerprint (17.5) and ML Model (15.5) and Sightengine (25.0) are all below 70 → -15 penalty.

Final Trust Score: 23.1 → Label: "Deepfake"

4. ML Model — EfficientNet-B0

4.1 Architecture

```
Input: RGB Image (224 x 224 pixels)
         |
         v
+--------------------------+
| EfficientNet-B0 Backbone  |  <-- Pretrained on ImageNet (1000 classes)
| (Feature Extractor)       |      Extracts 1280 visual features
| - 16 MBConv Blocks        |
| - Squeeze-and-Excitation  |
+--------------------------+
         |
         v  (1280 features)
+--------------------------+
| Custom Classifier Head    |
|                           |
| Dropout(p=0.3)            |  <-- Prevents overfitting
| Linear(1280 -> 128)       |  <-- Dimensionality reduction
| ReLU                      |  <-- Non-linear activation
| Dropout(p=0.2)            |
| Linear(128 -> 2)          |  <-- 2 output classes
+--------------------------+
         |
         v
   Softmax -> [P(fake), P(real)]
```

4.2 How Prediction Works

- Resize input image to 224x224 pixels

- Normalize pixel values using ImageNet statistics:
  - Mean: [0.485, 0.456, 0.406] (per RGB channel)
  - Std: [0.229, 0.224, 0.225]
- Forward pass through the network
- Softmax converts raw logits to probabilities
- P(real) is extracted and multiplied by 100 to get the ML trust score

4.3 Video Prediction

For video files:

- Extract up to 12 evenly-spaced frames from the video
- Run each frame through the model independently
- Average all frame probabilities to get the final video trust score

This catches deepfakes where only some frames are manipulated (temporal inconsistency).

4.4 Training Details

Parameter	Value
Base Architecture	EfficientNet-B0 (pretrained on ImageNet)
Training Dataset	200 images (100 fake + 100 real)
Fake Sources	StyleGAN via thispersondoesnotexist.com (1024x1024)
Real Sources	Royalty-free photographs from Unsplash (640x640 crops)
Train/Val Split	80% training / 20% validation
Validation Accuracy	95%
Discrimination Gap	81.2% (fake avg 15.5% vs real avg 96.8%)
Real Recall	100% (zero false accusations on real images)
Optimizer	AdamW (lr=1e-4, weight_decay=1e-4)
Loss Function	CrossEntropyLoss with label smoothing (0.1)
Data Augmentation	Random crop, horizontal flip, rotation, color jitter

4.5 Why EfficientNet-B0?

EfficientNet-B0 was chosen because:

- Compound scaling — balances network depth, width, and resolution efficiently
- Small but powerful — only 5.3M parameters (fast inference, low memory)
- Transfer learning — pretrained on ImageNet gives strong visual feature extraction even with a small training dataset
- Proven for deepfake detection — widely used in academic deepfake detection research

5. Sightengine API Integration

5.1 How It Works

Sightengine is a third-party AI content moderation API. We use their deepfake detection model:

- Send image/video to https://api.sightengine.com/1.0/check.json
- API returns: { "type": { "deepfake": 0.85 } } where 0 = authentic, 1 = deepfake
- We convert: trust_score = (1 - deepfake_score) x 100

5.2 Why Use a Third-Party API?

- Trained on millions of images — much larger training set than our 200-image dataset
- Regularly updated — Sightengine updates their model as new deepfake techniques emerge
- Provides a second opinion — our ML model and Sightengine are independent detectors; agreement between them increases confidence
- Handles edge cases — catches deepfakes that our model might miss (and vice versa)

5.3 Fallback Behavior

If Sightengine API is unavailable (no credentials, network error, API down):

- Sightengine weight (20%) is redistributed to other analyzers
- A warning anomaly is added to the result
- If ML model is also unavailable, trust score is capped at 65

6. Heuristic Analyzers

6.1 Metadata Analyzer

File: backend/services/metadata_analyzer.py Weight: 8% of composite score Purpose: Analyzes file metadata (EXIF) for tampering indicators

What It Checks

For Images:

Check	How	Penalty
Missing Camera Info	Looks for EXIF fields: Make, Model, LensModel	-20 points
Irregular Timestamps	Compares DateTimeOriginal vs DateTime	-30 points
Suspicious Compression	file_size / (width x height x 3) ratio analysis	-25 points
Editing Software	Checks Software field for Photoshop, GIMP, etc.	Flagged in details
Multi-flag	2 or more of the above	Additional -10 points

For Videos:

- Non-standard frame rates (not 24/25/30/60 fps) triggers irregular timestamp flag
- Very low resolution (<360p) triggers suspicious compression flag
- Codec analysis

For Audio:

- Lossy format detection (MP3, AAC, OGG)
- Very small file size triggers heavy compression flag

Important: Why Missing EXIF Is Not Conclusive

Many legitimate images have no EXIF data:

- Social media platforms (Instagram, WhatsApp, Twitter) strip EXIF on upload
- Stock photo sites (Unsplash, Pexels) often remove EXIF
- Screenshots have no camera metadata
- Web-downloaded images typically lack EXIF

Therefore, missing EXIF receives only a moderate penalty (-20 out of 100) on the metadata sub-score, which contributes only 8% to the final composite. Net effect: ~1.6 point reduction in the final trust score. The system relies more heavily on the ML model and frequency analysis.

6.2 Fingerprint Analyzer

File: backend/services/fingerprint_analyzer.py Weight: 12% of composite score Purpose: Identifies which deepfake tool was used by analyzing unique artifacts

Three Analysis Techniques

a) Frequency-Domain Analysis (FFT — Fast Fourier Transform)

```
Image -> Grayscale -> 2D FFT -> Magnitude Spectrum -> Radial Profile -> Autocorrelation
```

- Converts the image from spatial domain to frequency domain
- GAN-generated images have periodic peaks in the frequency spectrum caused by the neural network's upsampling layers (transposed convolutions)
- Natural images have smooth, monotonically decreasing frequency distribution
- If autocorrelation of radial profile shows >2 secondary peaks with correlation >0.3 then GAN fingerprint is detected

b) Face Boundary Analysis

- Detects faces using OpenCV's Haar Cascade classifier
- Analyzes the edge region around the face (5-pixel dilated border minus 5-pixel eroded border)
- Computes Laplacian variance (measure of sharpness)
- Low variance (<100) at face boundary = blurring artifact from face-swap blending
- This is a classic indicator of FaceSwap and DeepFaceLab manipulations

c) GAN Dimension and Metadata Indicators

GAN Architecture	Typical Output Size
StyleGAN / StyleGAN2	1024x1024
DALL-E, Midjourney	1024x1024
StyleGAN2 (lower config)	512x512
ProGAN, early GANs	256x256

When all three conditions are met — GAN-typical dimensions + no camera EXIF + high JPEG quality — the probability of GAN origin is very high (+55 score).

Known Tool Detection

Deepfake Tool	Detection Signals	Score Boost
StyleGAN	Periodic spectral peaks + GAN dimensions + no EXIF + high quality	Up to +95
FaceSwap	Low high-frequency ratio + face boundary blur	Up to +55
DeepFaceLab	Face boundary blur + mask edge artifacts	Up to +30
NeuralTextures	High-frequency energy ratio > 0.7	Up to +30
Face2Face	Expression artifacts + temporal flicker	Via other signals

A tool is named in the result only if its score exceeds 30 (to avoid false positives).

6.3 Compression Analyzer

File: backend/services/compression_analyzer.py Weight: 8% of composite score Purpose: Detects compression artifacts and identifies social media platform origin

JPEG Blocking Artifact Detection

JPEG compression divides images into 8x8 pixel blocks. When an image is re-compressed (common in deepfakes shared on social media), the block boundaries become visible:

- Measure pixel difference at every 8-pixel boundary (horizontal and vertical)
- Measure pixel difference at non-boundary positions
- Blockiness score = (boundary_diff - non_boundary_diff) / non_boundary_diff
- Score > 0.3 = significant artifacts, > 0.1 = mild artifacts

Social Media Platform Detection

Platform	Width Range	Quality Range
WhatsApp	600-1600px	30-60% quality
Instagram	600-1080px	40-75% quality
TikTok	540-1080px	50-80% quality
Twitter/X	600-4096px	60-85% quality
Telegram	1280-2560px	70-90% quality

If image dimensions and estimated JPEG quality match a platform's signature, the platform is identified in the report.

6.4 Audio Analyzer

File: backend/services/audio_analyzer.py Weight: 4% of composite score Applies To: Audio and Video files only Library: librosa (Python audio analysis)

Five Detection Features

Feature	What It Measures	Suspicious When
Spectral Flatness	How noise-like vs tonal the audio is	> 0.4 (too uniform = synthet..
Zero-Crossing Rate (std)	Variation in waveform zero crossings	< 0.01 (unnaturally uniform)
MFCC Variance	Mel-frequency cepstral coefficient variation acro..	< 5.0 (low variation = synth..
Pitch Stability (std)	How much the fundamental frequency varies	< 10 Hz (unnaturally stable ..
Spectral Rolloff	Where 85% of frequency energy lies	< 2000 Hz (limited frequency..

Clone Score Calculation:

```
clone_score = (number_of_flags x 20) + spectral_flatness_bonus
```

- Score > 50 is classified as "cloned" voice
- Trust conversion: trust = 100 - clone_score

Why These Features Work

Voice cloning systems (like Tacotron, VALL-E, or Bark) generate audio by synthesizing waveforms from text or reference audio. They typically produce:

- More uniform spectral characteristics than natural speech (constant quality across frames)
- Less pitch variation than natural human speech (humans naturally vary pitch)
- Smoother MFCC contours (natural speech has high MFCC variation due to consonants, breathing, emphasis)

6.5 Emotion Mismatch Analyzer

File: backend/services/emotion_analyzer.py Weight: 4% of composite score Applies To: Video files primarily

How It Works

- Facial Emotion Detection: Uses OpenCV Haar Cascade for face detection, then analyzes pixel intensity statistics of the face region:
  - High std deviation (>60) = "surprised"
  - Low mean (<80) = "sad"
  - High mean (>160) = "happy"
  - Otherwise = "neutral"
- Audio Emotion Detection: Uses librosa to extract RMS energy, zero-crossing rate, and spectral centroid:
  - High energy + high spectral centroid = "angry"
  - High energy + high ZCR = "happy"
  - Very low energy = "sad"
  - High spectral centroid = "surprised"
  - Otherwise = "neutral"
- Mismatch Scoring:
  - Emotions match = 5% mismatch (very low suspicion)
  - Emotions don't match = 75% mismatch (high suspicion)
  - Unknown emotion = 30% mismatch (uncertain)

Rationale: In a genuine video, facial expressions should correlate with vocal tone. A deepfake that replaces the face but keeps the original audio may show a happy face with an angry voice.

6.6 Lip-Sync Analyzer

File: backend/services/sync_analyzer.py Weight: 4% of composite score Applies To: Video files only

How It Works

- Mouth Motion Detection:
  - Detect face in each frame using Haar Cascade
  - Extract lower 40% of face (mouth region)
  - Calculate frame-to-frame pixel difference in mouth region
  - Produces a motion magnitude time series
- Audio Onset Detection:
  - Uses librosa's onset detection to find when speech segments begin
  - Calculates onset density (onsets per second)
- Correlation Analysis:

Condition	Mismatch Score	Interpretation
High audio activity + low mouth motion	80%	Talking audio but still mouth — likely dubbed
Low audio activity + high mouth motion	70%	Moving mouth but no speech — possible manipulation
Normal correlation	0-50%	Lip movement matches speech pattern

---

7. Trust Score Thresholds and Labels

Score Range	Label	Color	Meaning
70 — 100	Authentic	Green	Multiple analyzers agree the media is genuine
40 — 69	Suspicious	Yellow/Orange	Mixed signals — requires human investigation
0 — 39	Deepfake	Red	Strong evidence of manipulation or AI generation

Why These Thresholds?

- 70 (Authentic threshold): Requires a clear majority of weighted analyzers to agree the media is genuine. With the ML model at 40% weight, even a perfect ML score (100) only contributes 40 points — other analyzers must also support authenticity.
- 40 (Deepfake threshold): Below this, enough signals indicate manipulation to warrant a "Deepfake" classification. The multi-flag penalty system means that widespread suspicion across analyzers pushes scores below this threshold.
- 40-69 (Suspicious zone): This range exists because real-world media often has some anomalies (re-compression, missing EXIF, platform sharing) without being deepfakes. The system avoids false accusations by requiring strong evidence before labeling something "Deepfake."

8. Grad-CAM Heatmap (Explainable AI)

What Is Grad-CAM?

Gradient-weighted Class Activation Mapping (Grad-CAM) is an explainability technique that shows which regions of an image were most important for the model's decision.

How It Works (Step by Step)

- Forward pass: Run the image through EfficientNet-B0
- Hook into last convolutional layer: Capture the feature maps (1280 channels of 7x7 spatial resolution)
- Backward pass: Compute gradients of the fake class score with respect to the feature maps
- Weight calculation: Average the gradients spatially — one weight per channel
- Weighted combination: Multiply each feature map by its weight and sum
- ReLU: Remove negative activations (we only care about features that increase the fake score)
- Normalize and resize: Scale to 0-1 range, resize to original image dimensions
- Overlay: Apply JET colormap and blend 50/50 with original image

Interpreting the Heatmap

Color	Meaning
Red / Hot	Regions the model considers most likely manipulated
Yellow	Moderate manipulation evidence
Blue / Cold	Regions the model considers authentic

XAI Regions

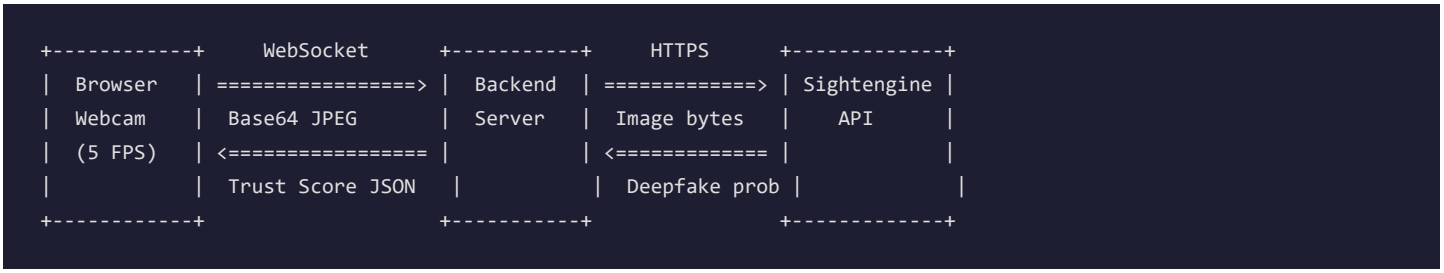
The system also extracts bounding boxes around high-activation areas (>50% activation threshold) and describes them:

- "High manipulation probability (85%) in Middle-Center area"
- "High manipulation probability (72%) in Top-Right area"

These help users understand specifically where the model detected potential manipulation.

9. Live Camera Monitoring

Architecture



How It Works

- Frontend captures webcam frames using browser's getUserMedia API
- Each frame is converted to base64-encoded JPEG
- Sent to backend via WebSocket connection at /api/live/ws
- Authentication: JWT token sent as first WebSocket message (not in URL for security)
- Backend decodes base64 to raw image bytes
- Sends image bytes to Sightengine API deepfake detection endpoint
- Sightengine returns deepfake_score (0 = authentic, 1 = deepfake)
- Backend converts: trust_score = (1 - deepfake_score) x 100
- Applies same thresholds: >= 70 Authentic, 40-69 Suspicious, <40 Deepfake
- Sends result back to frontend via WebSocket as JSON

Rate Limiting and Safety

Control	Value	Purpose
Max frame rate	5 FPS (200ms interval)	Prevents API rate limit exhaustion
Max payload size	1.5 MB per frame	Prevents memory exhaustion
WebSocket auth	JWT in first message	Prevents unauthorized access

Why Only Sightengine for Live Mode?

The full 8-analyzer pipeline requires:

- Saving the file to disk
- Running multiple analysis functions
- ML model inference

This adds 500ms+ latency per frame, making it unsuitable for real-time use. The Sightengine API responds in ~100-200ms, enabling near real-time analysis at 5 FPS.

10. AI Filters and Their Effect on Trust Score

Will AI Filters Reduce the Trust Score?

Yes. AI filters (beauty mode, face reshape, AR overlays) modify pixel patterns in ways that overlap with deepfake manipulation techniques:

Filter Type	What It Does to Pixels	Why It Triggers Detection
Beauty / Skin Smoothing	Reduces high-frequency textu..	Similar to GAN-generated smooth skin ..
Face Reshape (bigger eyes, slimme..	Warps facial geometry	Same geometric changes as face-swap d..
AR Overlays (dog ears, cartoon fa..	Blends artificial elements o..	Creates blending artifacts like face ..
Background Blur	Applies selective depth blur	Alters frequency-domain characteristics
Color Filters (warm, cool, vintage)	Shifts color distribution	May trigger compression/quality anoma..

Which Analyzers Are Affected?

- Sightengine API (20%): Trained to detect AI face manipulation — filters trigger similar features
- ML Model (40%): EfficientNet-B0 was trained on AI-generated faces; filtered faces share statistical properties
- Fingerprint Analyzer (12%): Filters alter the frequency spectrum, potentially creating patterns similar to GAN artifacts
- Metadata Analyzer (8%): Filtered images from apps often lack original camera EXIF data

Is This a Bug or a Feature?

It is correct behavior. The system detects media manipulation, not just deepfakes specifically. A heavily filtered image IS manipulated — the AI has altered the original pixel data. The trust score reflects the degree of manipulation present.

For forensic use cases, this is desirable: investigators need to know if media has been altered in any way, including filters.

11. Handling Images Without Camera Metadata

The Problem

Most images downloaded from the internet have no EXIF/camera metadata:

- Social media strips EXIF (Instagram, WhatsApp, Twitter)
- Stock photo sites may remove it (Unsplash, Pexels)
- Screenshots have no camera data
- AI-generated images never have real camera data

How MediaGuardX Handles This

Missing metadata is treated as a weak signal, not a conclusion.

Layer	Impact of Missing EXIF
Metadata Analyzer sub-score	-20 points (out of 100)
Metadata weight in composite	8%
Net effect on final trust score	~1.6 points
Other analyzers	Unaffected — ML model, Sightengine, fingerprint, compression all still..

Design Philosophy

The system was deliberately designed so that metadata analysis contributes only 8% to the final score. This prevents false accusations on legitimate web images that simply had their EXIF stripped.

The primary detectors — ML Model (40%) and Sightengine (20%) — analyze the actual pixel content of the image, not its metadata. A real photograph will score high on these analyzers regardless of whether EXIF data is present.

However, when missing metadata combines with other suspicious signals:

- Missing EXIF + GAN dimensions (1024x1024) + high quality = Fingerprint analyzer boosts StyleGAN probability by +55
- Missing EXIF + ML model says fake + Sightengine says fake = Multi-flag penalty applies (-15)

In this way, missing metadata becomes significant only when corroborated by other evidence.

12. Adaptive Learning System

Overview

MediaGuardX includes an adaptive learning feature that allows the model to improve over time based on user feedback.

How It Works

```
User submits feedback ("This was actually real/fake")
         |
         v
Image copied to adaptive_data/real/ or adaptive_data/fake/
         |
         v
When 10+ samples accumulated, Admin triggers retraining
         |
         v
Fine-tune existing model with very low learning rate (1e-5)
         |
         v
New model hot-swapped in memory (no restart needed)
```

Retraining Parameters

Parameter	Value
Starting point	Current production model
New data	User feedback samples (saved in adaptive_data/)
Learning rate	1e-5 (very low — prevents catastrophic forgetting)
Epochs	5
Optimizer	AdamW (weight_decay=1e-4)
Loss	CrossEntropyLoss with label smoothing (0.1)
Data augmentation	Random crop, flip, rotation, color jitter
Minimum samples	10 before retraining is allowed
Model backup	Automatic .pth.bak before overwrite

Why Adaptive Learning Matters

- Domain adaptation: If the system is deployed in an environment with specific types of media (e.g., surveillance cameras, social media), user feedback tunes the model to that domain
- New deepfake techniques: As new generation tools emerge, user feedback on new types of fakes helps the model learn to detect them
- Reduced false positives: If the model consistently misclassifies certain real images, feedback corrects this

13. Security Architecture

Authentication

- Supabase Auth with JWT tokens
- Row-Level Security (RLS) on all database tables
- Role-based access: user, investigator, admin

- Deactivated user enforcement (is_active check)

API Security

- SSRF Protection: URL scheme validation, DNS resolution, private IP rejection, safe redirect handling
- Rate Limiting: Per-minute limits via slowapi
- Streaming File Upload: 8KB chunks with early abort on size violation
- File Path Confinement: Path.resolve() validation within upload directory
- WebSocket Auth: First-message JWT protocol (tokens not exposed in URL)
- Model Loading Security: weights_only=True prevents arbitrary code execution

Data Security

- Error details hidden in production (generic error messages)
- Authorization checks on all detection access (owner, investigator, or admin only)
- Sensitive credentials stored in .env files (gitignored)

14. Technology Stack

Backend

Component	Technology
Framework	FastAPI (Python 3.12)
Database	Supabase (PostgreSQL)
Authentication	Supabase Auth (JWT)
ML Framework	PyTorch 2.0+
Image Processing	OpenCV, Pillow (PIL)
Audio Processing	librosa
External API	Sightengine
WebSocket	FastAPI native WebSocket support

Frontend

Component	Technology
Framework	React 18 + TypeScript
Build Tool	Vite
State Management	Zustand
Routing	React Router v6
UI Components	Tailwind CSS
Auth Client	Supabase JS Client

ML Model

Component	Detail
Architecture	EfficientNet-B0
Parameters	5.3M (backbone) + custom classifier head
Input	224x224 RGB
Output	2 classes (fake, real)
Training Data	200 images (100 fake + 100 real)
Accuracy	95% validation
Inference	CPU or CUDA GPU

Document generated for MediaGuardX v2.0.0 — Supabase Architecture